SERVICIO NACIONAL DE APRENDIZAJE – SENA
Tecnología en Análisis y Desarrollo de Software (228118)

DISEÑO Y DESARROLLO DE SERVICIOS WEB – PROYECTO

Evidencia GA7-220501096-AA5-EV03

Documentación de la API REST de la Plataforma Digital Integral para la Gestión de Proyectos de Impacto Ambiental

Aprendices:
Roycer Cervantes
Sebastián Rodríguez

Ficha: _____
Instructor(a): _____

Santa Marta, Magdalena
Octubre de 2026

Contenido

1. Introducción....3
2. Objetivos....3
2.1 Objetivo general....3
2.2 Objetivos específicos....3
3. Alcance y arquitectura de la API....4
3.1 Qué resuelve la API....4
3.2 Tecnologías seleccionadas....4
3.3 Arquitectura por capas....4
3.4 Estructura de carpetas....5
4. Modelo de datos....5
4.1 usuarios....6
4.2 proyectos....6
4.3 impactos_ambientales....6
4.4 actividades....7
4.5 monitoreos....7
5. Seguridad y permisos....7
6. Convenciones generales de los servicios....8
6.1 URL base y formato....8
6.2 Formato de respuesta....8
6.3 Códigos de estado HTTP....9
6.4 Paginación....9
6.5 Resumen de servicios....9
7. Documentación de cada servicio....11
7.1 Salud....11
7.2 Autenticación....11
7.3 Usuarios....15
7.4 Proyectos....19
7.5 Impactos ambientales....27
7.6 Actividades....32
7.7 Monitoreos....36
7.8 Reportes....41
8. Verificación de los servicios....43
9. Instalación y ejecución....44
10. Control de versiones....44
11. Conclusiones....44
12. Referencias....45

1. Introducción

En las evidencias anteriores de esta guía definimos los componentes del front-end en React (Navbar, Dashboard, ProjectList, ProjectForm, ImpactChart, entre otros) y construimos un primer servicio de registro e inicio de sesión (AA5-EV01). Faltaba la pieza que conecta todo: los servicios web que le entregan los datos reales al front. Este documento presenta precisamente eso, la API REST de nuestra Plataforma Digital Integral para la Gestión de Proyectos de Impacto Ambiental.

La idea de la plataforma nace de algo que vemos seguido en la región: los proyectos que afectan el ambiente (obras de saneamiento, restauraciones de manglar, monitoreos de ríos) suelen manejar su información en hojas de cálculo sueltas, correos y documentos que nadie encuentra cuando se necesitan. Con la API, esa información queda centralizada: quién es el responsable de cada proyecto, qué impactos se identificaron y qué tan graves son, qué actividades de manejo están pendientes o vencidas, y si las mediciones de calidad del agua o del aire cumplen o no con los límites permisibles.

Construimos la API con Node.js, Express y MySQL, siguiendo una arquitectura por capas (rutas, controladores y modelos), con autenticación por token JWT y permisos por rol. En total quedaron 34 operaciones que cubren el CRUD completo de usuarios, proyectos, impactos, actividades y monitoreos, más dos servicios de reportes para el dashboard. Cada servicio está documentado en este informe con su método, URL, parámetros, permisos y un ejemplo real de petición y respuesta tomado de las pruebas que hicimos.

2. Objetivos

2.1 Objetivo general

Diseñar, codificar y documentar los servicios web de la Plataforma Digital Integral para la Gestión de Proyectos de Impacto Ambiental, de acuerdo con las características del software definidas en los artefactos del proyecto formativo, usando herramientas de control de versiones.

2.2 Objetivos específicos

- Diseñar el modelo de datos y los recursos REST que necesita el front-end en React para funcionar con datos reales.
- Codificar los servicios de autenticación, usuarios, proyectos, impactos ambientales, actividades, monitoreos y reportes aplicando los estándares de codificación definidos en la evidencia AA1-EV02.
- Proteger los servicios con autenticación JWT, contraseñas cifradas y permisos según el rol del usuario.
- Validar los datos de entrada (textos, números, fechas, correos, longitudes y valores permitidos) y responder errores claros.
- Documentar cada servicio con su método, URL, parámetros, permisos, respuestas y códigos de estado, tanto en este documento como en Swagger (OpenAPI 3).
- Versionar el proyecto con Git y publicarlo en un repositorio remoto de GitHub.

3. Alcance y arquitectura de la API

3.1 Qué resuelve la API

La API atiende los requerimientos que levantamos en las historias de usuario del proyecto. En resumen:

Necesidad del usuario	Servicios que la resuelven
Crear una cuenta e ingresar a la plataforma	POST /auth/registro, POST /auth/login, GET /auth/perfil
El administrador controla quién entra y con qué permisos	/usuarios (listar, ver, editar, cambiar rol, desactivar, activar)
Registrar y hacer seguimiento a los proyectos ambientales	/proyectos (CRUD, filtros, cambio de estado)
Identificar y valorar los impactos de cada proyecto	/proyectos/:id/impactos y /impactos/:id
Planear y controlar las actividades del plan de manejo	/proyectos/:id/actividades y /actividades/:id
Registrar mediciones y saber si cumplen la norma	/proyectos/:id/monitoreos y /monitoreos/:id
Ver indicadores en el dashboard	/reportes/dashboard y /reportes/proyectos/:id

3.2 Tecnologías seleccionadas

Tecnología	Uso	Por qué la escogimos
Node.js 18+	Entorno de ejecución	Usamos JavaScript tanto en el front (React) como en el back, así no cambiamos de lenguaje.
Express 4	Framework web	Es liviano, muy documentado y es el que vimos en el componente "Construcción de API".
MySQL 8	Base de datos	Ya teníamos el modelo relacional del proyecto y lo usamos en la evidencia AA5-EV01.
mysql2	Conector a MySQL	Soporta promesas (async/await) y consultas con parámetros.
jsonwebtoken (JWT)	Sesiones	El front guarda el token y lo envía en cada petición; el servidor no tiene que guardar sesiones.
bcryptjs	Contraseñas	Cifra las contraseñas con hash y sal; nunca se guardan en texto plano.
Swagger UI + OpenAPI 3	Documentación	Permite ver y probar los servicios desde el navegador en /api/docs.
Git + GitHub	Versionamiento	Control de cambios y trabajo en pareja (AA1).

3.3 Arquitectura por capas

Organizamos el código en capas para que cada archivo tenga una sola responsabilidad. Una petición recorre este camino:

```
Cliente (React / Postman)
      |  HTTP + JSON
      v
Rutas (src/routes)            -> define URL, método y middlewares
      v
Middlewares                   -> verificarToken, permitirRoles
      v
Controladores (src/controllers) -> valida datos, aplica reglas de negocio
      v
Modelos (src/models)          -> consultas SQL con parámetros
      v
MySQL (impacto_ambiental_db)

Cualquier error -> manejadorErrores -> { ok: false, mensaje, errores }
```

Gracias a esta separación, los controladores nunca escriben SQL y los modelos nunca responden HTTP. Si más adelante cambiamos de motor de base de datos, solo hay que tocar la carpeta models.

3.4 Estructura de carpetas

```
plataforma-impacto-ambiental-api/
├── database/
│   ├── schema.sql            tablas, llaves foráneas e índices
│   └── seed.js               datos de prueba
├── docs/
│   ├── openapi.yaml          documentación OpenAPI (Swagger)
│   └── ejemplos-respuestas.json
├── pruebas/probarApi.js      prueba automática de los 34 servicios
├── src/
│   ├── config/db.js          pool de conexiones a MySQL
│   ├── controllers/          authController, proyectoController, ...
│   ├── middlewares/          autenticacion, manejoErrores, envolverAsync
│   ├── models/               UsuarioModelo, ProyectoModelo, ...
│   ├── routes/               authRutas, proyectoRutas, ...
│   ├── utils/                validaciones, matrizImpacto, paginacion, ...
│   ├── validators/reglas.js  reglas de validación por recurso
│   ├── app.js                configuración de Express
│   └── server.js             arranque del servidor
├── .env.example
├── package.json
└── README.md
```

4. Modelo de datos

La base de datos impacto_ambiental_db tiene cinco tablas relacionadas. Un usuario puede ser responsable de muchos proyectos, y cada proyecto tiene muchos impactos, actividades y

monitoreos. Las tablas hijas usan ON DELETE CASCADE para que al eliminar un proyecto no queden registros huérfanos.

4.1 usuarios

Campo	Tipo	Descripción
id	INT PK AI	Identificador
nombre	VARCHAR(100)	Nombre completo
correo	VARCHAR(120) UNIQUE	Correo de acceso
password_hash	VARCHAR(255)	Contraseña cifrada con bcrypt
rol	ENUM('admin','gestor','consultor')	Rol del usuario
activo	TINYINT(1)	1 activo, 0 desactivado
creado_en / actualizado_en	DATETIME	Auditoría

4.2 proyectos

Campo	Tipo	Descripción
id	INT PK AI	Identificador
codigo	VARCHAR(20) UNIQUE	PIA-AAAA-NNNN, lo genera la API
nombre / descripcion	VARCHAR(150) / TEXT	Datos generales
tipo	ENUM	infraestructura, restauracion, conservacion, saneamiento, energia, educacion, otro
municipio / departamento	VARCHAR(80)	Ubicación
latitud / longitud	DECIMAL(9,6)	Coordenadas
fecha_inicio / fecha_fin	DATE	Duración
presupuesto	DECIMAL(15,2)	En pesos colombianos
estado	ENUM	planeacion, ejecucion, finalizado, suspendido
responsable_id	INT FK → usuarios	Gestor responsable

4.3 impactos_ambientales

Campo	Tipo	Descripción
id / proyecto_id	INT PK / FK	Identificador y proyecto
componente	ENUM	agua, aire, suelo, flora, fauna, paisaje, social

Campo	Tipo	Descripción
descripcion	VARCHAR(255)	Qué impacto es
naturaleza	ENUM	positivo o negativo
magnitud / probabilidad	TINYINT	Escala de 1 a 10
importancia	TINYINT	magnitud × probabilidad (calculado)
nivel	ENUM	bajo, moderado, alto, critico (calculado)
medida_manejo	TEXT	Cómo se previene, mitiga o compensa

4.4 actividades

Campo	Tipo	Descripción
id / proyecto_id	INT PK / FK	Identificador y proyecto
titulo / descripcion	VARCHAR(150) / TEXT	Qué hay que hacer
fecha_limite	DATE	Fecha de entrega
estado	ENUM	pendiente, en_progreso, completada
responsable_id	INT FK → usuarios	Quién la hace (opcional)

4.5 monitoreos

Campo	Tipo	Descripción
id / proyecto_id	INT PK / FK	Identificador y proyecto
parametro / unidad	VARCHAR	Ej.: Oxígeno disuelto, mg/L
valor	DECIMAL(12,4)	Valor medido
limite_permisible	DECIMAL(12,4)	Límite de referencia (opcional)
tipo_limite	ENUM	maximo o minimo
cumple_norma	TINYINT(1)	Calculado por la API (nulo si no hay límite)
punto_muestreo / fecha_muestreo	VARCHAR / DATE	Dónde y cuándo
observaciones	TEXT	Notas de campo

5. Seguridad y permisos

Todas las rutas, salvo /salud, /auth/registro y /auth/login, exigen el encabezado Authorization: Bearer <token>. El middleware verificarToken valida la firma del token y vuelve a consultar el usuario en la base de datos, de modo que si el administrador lo desactiva, pierde el acceso de inmediato aunque su token no haya expirado.

Los permisos dependen del rol y, para modificar un proyecto o algo que cuelgue de él, de si el usuario es su responsable:

Acción	admin	gestor	consultor
Ver proyectos, impactos, actividades, monitoreos y reportes	Sí	Sí	Sí
Crear proyectos	Sí	Sí (queda como responsable)	No
Modificar o eliminar un proyecto y sus registros	Sí, todos	Solo los suyos	No
Reabrir un proyecto finalizado	Sí	No	No
Administrar usuarios (listar, roles, desactivar)	Sí	No	No
Ver y editar su propia cuenta	Sí	Sí	Sí

Otras medidas que aplicamos:

- Contraseñas cifradas con bcrypt (10 rondas). Deben tener mínimo 8 caracteres, una letra y un número.
- El login responde el mismo mensaje si el correo no existe o si la contraseña está mal, para no dar pistas a quien intente adivinar cuentas.
- Todas las consultas SQL usan parámetros (?), lo que evita la inyección SQL.
- En las actualizaciones solo se aceptan columnas permitidas, así nadie puede cambiar su rol mandando "rol" en el PUT de su perfil.
- Los datos sensibles (clave JWT, contraseña de MySQL) van en el archivo .env, que no se sube al repositorio.

6. Convenciones generales de los servicios

6.1 URL base y formato

URL base en desarrollo: **http://localhost:3000/api**. Todas las peticiones y respuestas usan JSON (Content-Type: application/json). Las fechas se envían y se reciben con el formato AAAA-MM-DD.

6.2 Formato de respuesta

Todas las respuestas tienen la misma forma. Esto le facilita el trabajo al front, que siempre sabe dónde buscar los datos o el error:

```
// Respuesta correcta
{ "ok": true, "mensaje": "Proyecto creado correctamente", "datos": { ... } }

// Respuesta con error
{ "ok": false, "mensaje": "Hay datos inválidos en la petición",
  "errores": [ { "campo": "fechaFin", "mensaje": "La fecha de fin no puede ser
anterior..." } ] }
```

6.3 Códigos de estado HTTP

Código	Significado en la API
200 OK	Consulta, actualización o eliminación correcta
201 Created	Registro creado
400 Bad Request	Datos inválidos, id mal escrito o JSON mal formado
401 Unauthorized	Sin token, token inválido/expirado o credenciales incorrectas
403 Forbidden	El rol no tiene permiso o el usuario no es el responsable
404 Not Found	El recurso o la ruta no existe
409 Conflict	Correo ya registrado
500 Internal Server Error	Error inesperado (se registra en la consola del servidor)
503 Service Unavailable	La base de datos no responde (solo en /salud)

6.4 Paginación

Los listados de usuarios y proyectos aceptan ?pagina=1&limite=10 (máximo 50 por página) y devuelven, junto a los datos, el objeto paginacion con total, pagina, limite y totalPaginas.

6.5 Resumen de servicios

#	Método	URL	Servicio
1	GET	/salud	Estado de la API
2	POST	/auth/registro	Registro de usuario
3	POST	/auth/login	Inicio de sesión
4	GET	/auth/perfil	Perfil del usuario
5	GET	/usuarios	Listar usuarios
6	GET	/usuarios/:id	Consultar usuario
7	PUT	/usuarios/:id	Actualizar usuario
8	PATCH	/usuarios/:id/rol	Cambiar rol
9	DELETE	/usuarios/:id	Desactivar usuario
10	PATCH	/usuarios/:id/activar	Reactivar usuario
11	GET	/proyectos	Listar proyectos
12	POST	/proyectos	Crear proyecto
13	GET	/proyectos/:id	Consultar proyecto
14	PUT	/proyectos/:id	Actualizar proyecto
15	PATCH	/proyectos/:id/estado	Cambiar estado del proyecto

#	Método	URL	Servicio
16	DELETE	/proyectos/:id	Eliminar proyecto
17	GET	/proyectos/:proyectoId/impactos	Listar impactos de un proyecto
18	POST	/proyectos/:proyectoId/impactos	Registrar impacto
19	GET	/impactos/:id	Consultar impacto
20	PUT	/impactos/:id	Actualizar impacto
21	DELETE	/impactos/:id	Eliminar impacto
22	GET	/proyectos/:proyectoId/actividades	Listar actividades
23	POST	/proyectos/:proyectoId/actividades	Crear actividad
24	GET	/actividades/:id	Consultar actividad
25	PUT	/actividades/:id	Actualizar actividad
26	PATCH	/actividades/:id/estado	Cambiar estado de actividad
27	DELETE	/actividades/:id	Eliminar actividad
28	GET	/proyectos/:proyectoId/monitoreos	Listar monitoreos
29	POST	/proyectos/:proyectoId/monitoreos	Registrar monitoreo
30	GET	/monitoreos/:id	Consultar monitoreo
31	PUT	/monitoreos/:id	Actualizar monitoreo
32	DELETE	/monitoreos/:id	Eliminar monitoreo
33	GET	/reportes/dashboard	Resumen general (Dashboard)
34	GET	/reportes/proyectos/:id	Resumen de un proyecto

7. Documentación de cada servicio

A continuación se describe cada servicio. Los ejemplos de petición y respuesta no son inventados: se tomaron de la ejecución del script de pruebas (npm run probar) contra la API con los datos de prueba cargados. En los listados largos se muestra solo el primer registro y en las respuestas de login el token aparece recortado.

7.1 Salud

7.1.1 Estado de la API

Método	GET
URL	/api/salud
Descripción	Informa si la API está encendida y si logra conectarse a MySQL. La usamos para descartar problemas de conexión antes de probar lo demás.
Acceso	Público
Parámetros	Ninguno
Respuesta exitosa	200
Errores posibles	503: la API responde pero MySQL no.

Ejemplo de petición

```
GET /api/salud
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "API Plataforma de Gestión de Proyectos de Impacto Ambiental",
  "version": "1.0.0",
  "baseDatos": "conectada",
  "fecha": "2026-10-23T03:44:07.104Z"
}
```

7.2 Autenticación

7.2.1 Registro de usuario

Método	POST
URL	/api/auth/registro
Descripción	Crea una cuenta nueva y devuelve de una vez el token para que el usuario quede con la sesión iniciada. La contraseña se guarda cifrada con bcrypt.
Acceso	Público
Parámetros	Body: nombre (3-100), correo (válido, único), password (mín. 8, letra y

	número), rol (gestor | consultor, por defecto consultor).
Respuesta exitosa	201
Errores posibles	400 datos inválidos · 409 correo ya registrado

Ejemplo de petición

```
POST /api/auth/registro
{
  "nombre": "Laura Mendoza",
  "correo": "laura.mendoza.1792727047113@correo.com",
  "password": "Manglar2026",
  "rol": "gestor"
}
```

Respuesta obtenida (201)

```
{
  "ok": true,
  "mensaje": "Usuario registrado correctamente",
  "datos": {
    "usuario": {
      "id": 5,
      "nombre": "Laura Mendoza",
      "correo": "laura.mendoza.1792727047113@correo.com",
      "rol": "gestor",
      "activo": true,
      "creadoEn": "2026-10-23 03:44:07",
      "actualizadoEn": "2026-10-23 03:44:07"
    },
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  }
}
```

Ejemplo de error (400)

```
POST /api/auth/registro
{"nombre":"Al","correo":"correo-malo","password":"123","rol":"admin"}

{
  "ok": false,
  "mensaje": "Hay datos inválidos en la petición",
  "errores": [
    {
      "campo": "nombre",
      "mensaje": "El campo nombre debe tener mínimo 3 caracteres"
    },
    {
      "campo": "correo",
      "mensaje": "El campo correo debe ser un correo válido"
    },
    {
      "campo": "password",
      "mensaje": "El campo password debe tener mínimo 8 caracteres"
```

```
    },
    {
      "campo": "rol",
      "mensaje": "El campo rol solo acepta: gestor, consultor"
    }
  ]
}
```

7.2.2 Inicio de sesión

Método	POST
URL	/api/auth/login
Descripción	Valida correo y contraseña. Si coinciden responde "Autenticación satisfactoria" con un token JWT válido por 8 horas; si no, "Error en la autenticación".
Acceso	Público
Parámetros	Body: correo, password.
Respuesta exitosa	200
Errores posibles	400 faltan datos · 401 credenciales incorrectas · 403 cuenta desactivada

Ejemplo de petición

```
POST /api/auth/login
{
  "correo": "admin@plataforma.com",
  "password": "Ambiental2026"
}
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Autenticación satisfactoria",
  "datos": {
    "usuario": {
      "id": 1,
      "nombre": "Administrador General",
      "correo": "admin@plataforma.com",
      "rol": "admin",
      "activo": true,
      "creadoEn": "2026-10-23 03:44:05",
      "actualizadoEn": "2026-10-23 03:44:05"
    },
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  }
}
```

Ejemplo de error (401)

```
POST /api/auth/login
{"correo":"admin@plataforma.com","password":"claveEquivocada1"}

{
  "ok": false,
  "mensaje": "Error en la autenticación: correo o contraseña incorrectos"
}
```

7.2.3 Perfil del usuario

Método	GET
URL	/api/auth/perfil
Descripción	Devuelve los datos del dueño del token. El Navbar del front lo usa para mostrar el nombre y el rol.
Acceso	Token (cualquier rol)
Parámetros	Encabezado Authorization: Bearer <token>
Respuesta exitosa	200
Errores posibles	401 sin token, token inválido o expirado

Ejemplo de petición

```
GET /api/auth/perfil
Authorization: Bearer <token>
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Perfil del usuario autenticado",
  "datos": {
    "id": 2,
    "nombre": "Roycer Cervantes",
    "correo": "roycer.cervantes@plataforma.com",
    "rol": "gestor",
    "activo": true,
    "creadoEn": "2026-10-23 03:44:05",
    "actualizadoEn": "2026-10-23 03:44:05"
  }
}
```

Ejemplo de error (401)

```
GET /api/auth/perfil

{
  "ok": false,
  "mensaje": "Debes iniciar sesión. Falta el token de acceso"
}
```

7.3 Usuarios

7.3.1 Listar usuarios

Método	GET
URL	/api/usuarios
Descripción	Lista las cuentas con paginación y filtros.
Acceso	Token · solo admin
Parámetros	Query: rol, activo (true/false), buscar, pagina, limite (máx. 50).
Respuesta exitosa	200
Errores posibles	401 · 403 si no es admin

Ejemplo de petición

```
GET /api/usuarios?limite=5
Authorization: Bearer <token>
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Listado de usuarios",
  "datos": [
    {
      "id": 1,
      "nombre": "Administrador General",
      "correo": "admin@plataforma.com",
      "rol": "admin",
      "activo": true,
      "creadoEn": "2026-10-23 03:44:05",
      "actualizadoEn": "2026-10-23 03:44:05"
    },
    "... (4 registro(s) más)"
  ],
  "paginacion": {
    "total": 5,
    "pagina": 1,
    "limite": 5,
    "totalPaginas": 1
  }
}
```

Ejemplo de error (403)

```
GET /api/usuarios

{
  "ok": false,
  "mensaje": "No tienes permisos para realizar esta acción"
}
```

7.3.2 Consultar usuario

Método	GET
URL	/api/usuarios/:id
Descripción	Muestra los datos de un usuario.
Acceso	Token · admin o el mismo usuario
Parámetros	Ruta: id (entero positivo).
Respuesta exitosa	200
Errores posibles	400 id inválido · 403 · 404

Ejemplo de petición

```
GET /api/usuarios/5
Authorization: Bearer <token>
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Usuario encontrado",
  "datos": {
    "id": 5,
    "nombre": "Laura Mendoza",
    "correo": "laura.mendoza.1792727047113@correo.com",
    "rol": "gestor",
    "activo": true,
    "creadoEn": "2026-10-23 03:44:07",
    "actualizadoEn": "2026-10-23 03:44:07"
  }
}
```

Ejemplo de error (403)

```
GET /api/usuarios/5

{
  "ok": false,
  "mensaje": "Solo puedes consultar tu propia cuenta"
}
```

7.3.3 Actualizar usuario

Método	PUT
URL	/api/usuarios/:id
Descripción	Cambia nombre, correo o contraseña. Se envían solo los campos que cambian.

Acceso	Token · admin o el mismo usuario
Parámetros	Body (opcionales): nombre, correo, password.
Respuesta exitosa	200
Errores posibles	400 · 403 · 404 · 409 correo en uso

Ejemplo de petición

```
PUT /api/usuarios/5
{
  "nombre": "Laura Mendoza Pérez"
}
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Usuario actualizado correctamente",
  "datos": {
    "id": 5,
    "nombre": "Laura Mendoza Pérez",
    "correo": "laura.mendoza.1792727047113@correo.com",
    "rol": "gestor",
    "activo": true,
    "creadoEn": "2026-10-23 03:44:07",
    "actualizadoEn": "2026-10-23 03:44:07"
  }
}
```

7.3.4 Cambiar rol

Método	PATCH
URL	/api/usuarios/:id/rol
Descripción	Asigna otro rol. El admin no puede cambiarse su propio rol (para no quedarse sin administrador por error).
Acceso	Token · solo admin
Parámetros	Body: rol (admin | gestor | consultor).
Respuesta exitosa	200
Errores posibles	400 · 403 · 404

Ejemplo de petición

```
PATCH /api/usuarios/5/rol
{
  "rol": "consultor"
}
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Rol actualizado a consultor",
  "datos": {
    "id": 5,
    "nombre": "Laura Mendoza Pérez",
    "correo": "laura.mendoza.1792727047113@correo.com",
    "rol": "consultor",
    "activo": true,
    "creadoEn": "2026-10-23 03:44:07",
    "actualizadoEn": "2026-10-23 03:44:07"
  }
}
```

7.3.5 Desactivar usuario

Método	DELETE
URL	/api/usuarios/:id
Descripción	No borra la fila: pone activo = false. Así no se pierde el historial de responsables de los proyectos. Un usuario desactivado no puede iniciar sesión.
Acceso	Token · solo admin
Parámetros	Ruta: id.
Respuesta exitosa	200
Errores posibles	400 si es uno mismo o ya estaba desactivado · 404

Ejemplo de petición

```
DELETE /api/usuarios/5
Authorization: Bearer <token>
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Usuario desactivado correctamente",
  "datos": {
    "id": 5,
    "nombre": "Laura Mendoza Pérez",
    "correo": "laura.mendoza.1792727047113@correo.com",
    "rol": "consultor",
    "activo": false,
    "creadoEn": "2026-10-23 03:44:07",
    "actualizadoEn": "2026-10-23 03:44:07"
  }
}
```

Ejemplo de error (403)

```
POST /api/auth/login
{"correo":"laura.mendoza.1792727047113@correo.com","password":"Manglar2026"}

{
  "ok": false,
  "mensaje": "Tu cuenta está desactivada. Comunícate con el administrador"
}
```

7.3.6 Reactivar usuario

Método	PATCH
URL	/api/usuarios/:id/activar
Descripción	Vuelve a activar una cuenta desactivada.
Acceso	Token · solo admin
Parámetros	Ruta: id.
Respuesta exitosa	200
Errores posibles	403 · 404

Ejemplo de petición

```
PATCH /api/usuarios/5/activar
Authorization: Bearer <token>
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Usuario activado correctamente",
  "datos": {
    "id": 5,
    "nombre": "Laura Mendoza Pérez",
    "correo": "laura.mendoza.1792727047113@correo.com",
    "rol": "consultor",
    "activo": true,
    "creadoEn": "2026-10-23 03:44:07",
    "actualizadoEn": "2026-10-23 03:44:07"
  }
}
```

7.4 Proyectos

7.4.1 Listar proyectos

Método	GET
URL	/api/proyectos
Descripción	Lista los proyectos con paginación. Alimenta el componente ProjectList del

	front.
Acceso	Token (cualquier rol)
Parámetros	Query: estado, tipo, municipio, buscar (nombre, código o descripción), responsableId, pagina, limite.
Respuesta exitosa	200
Errores posibles	400 filtro con valor no permitido

Ejemplo de petición

```
GET /api/proyectos?limite=5
Authorization: Bearer <token>
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Listado de proyectos",
  "datos": [
    {
      "id": 2,
      "codigo": "PIA-2026-0002",
      "nombre": "Planta de tratamiento de aguas residuales del sector Gaira",
      "descripcion": "Construcción de una PTAR con lagunas de estabilización para
reducir vertimientos directos al mar.",
      "tipo": "saneamiento",
      "municipio": "Santa Marta",
      "departamento": "Magdalena",
      "latitud": 11.183,
      "longitud": -74.226,
      "fechaInicio": "2026-05-10",
      "fechaFin": "2027-12-15",
      "presupuesto": 1800000000,
      "estado": "planeacion",
      "responsableId": 3,
      "responsableNombre": "Sebastián Rodríguez",
      "creadoEn": "2026-10-23 03:44:05",
      "actualizadoEn": "2026-10-23 03:44:05"
    },
    "... (2 registro(s) más)"
  ],
  "paginacion": {
    "total": 3,
    "pagina": 1,
    "limite": 5,
    "totalPaginas": 1
  }
}
```

Ejemplo de error (400)

```
GET /api/proyectos?estado=terminado
```

```
{
  "ok": false,
  "mensaje": "El filtro estado solo acepta: planeacion, ejecucion, finalizado,
suspendido"
}
```

7.4.2 Crear proyecto

Método	POST
URL	/api/proyectos
Descripción	Registra un proyecto. La API genera el código PIA-AAAA-NNNN. Si lo crea un gestor, él queda como responsable; el admin puede asignarlo a otro gestor.
Acceso	Token · admin o gestor
Parámetros	Body: nombre*, tipo*, municipio*, departamento*, fechaInicio*, descripcion, latitud (-90 a 90), longitud (-180 a 180), fechaFin (≥ fechaInicio), presupuesto (≥ 0), estado, responsableId (solo admin).
Respuesta exitosa	201
Errores posibles	400 datos inválidos · 403 consultor

Ejemplo de petición

```
POST /api/proyectos
{
  "nombre": "Reforestación de la cuenca alta del río Gaira",
  "descripcion": "Siembra de especies nativas en zonas de recarga hídrica afectadas por
incendios.",
  "tipo": "restauracion",
  "municipio": "Santa Marta",
  "departamento": "Magdalena",
  "latitud": 11.112,
  "longitud": -74.071,
  "fechaInicio": "2026-11-01",
  "fechaFin": "2027-10-31",
  "presupuesto": 120000000
}
```

Respuesta obtenida (201)

```
{
  "ok": true,
  "mensaje": "Proyecto creado correctamente",
  "datos": {
    "id": 4,
    "codigo": "PIA-2026-0004",
    "nombre": "Reforestación de la cuenca alta del río Gaira",
    "descripcion": "Siembra de especies nativas en zonas de recarga hídrica afectadas
por incendios.",
    "tipo": "restauracion",
```

```
    "municipio": "Santa Marta",
    "departamento": "Magdalena",
    "latitud": 11.112,
    "longitud": -74.071,
    "fechaInicio": "2026-11-01",
    "fechaFin": "2027-10-31",
    "presupuesto": 120000000,
    "estado": "planeacion",
    "responsableId": 2,
    "responsableNombre": "Roycer Cervantes",
    "creadoEn": "2026-10-23 03:44:07",
    "actualizadoEn": "2026-10-23 03:44:07"
  }
}
```

Ejemplo de error (400)

```
POST /api/proyectos
{"nombre":"Reforestación de la cuenca alta del río Gaira","descripcion":"Siembra de
especies nativas en zonas de recarga hídrica afectadas por
incendios.","tipo":"mineria","municipio":"Santa
Marta","departamento":"Magdalena","latitud":200,"longitud":-74.071,"fechaInicio":"2026-
11-01","fechaFin":"2026-01-01","presupuesto":120000000}

{
  "ok": false,
  "mensaje": "Hay datos inválidos en la petición",
  "errores": [
    {
      "campo": "tipo",
      "mensaje": "El campo tipo solo acepta: infraestructura, restauracion,
conservacion, saneamiento, energia, educacion, otro"
    },
    {
      "campo": "latitud",
      "mensaje": "El campo latitud debe ser menor o igual a 90"
    }
  ]
}
```

7.4.3 Consultar proyecto

Método	GET
URL	/api/proyectos/:id
Descripción	Devuelve el proyecto junto con sus impactos, actividades y monitoreos en una sola respuesta (pantalla ProjectDetail).
Acceso	Token (cualquier rol)
Parámetros	Ruta: id.
Respuesta exitosa	200

Errores posibles	400 id inválido · 404

Ejemplo de petición

```
GET /api/proyectos/1
Authorization: Bearer <token>
```

Respuesta obtenida (200)


```
{
  "ok": true,
  "mensaje": "Proyecto encontrado",
  "datos": {
    "id": 1,
    "codigo": "PIA-2026-0001",
    "nombre": "Restauración de manglar en la Ciénaga Grande de Santa Marta",
    "descripcion": "Siembra de propágulos de mangle rojo y recuperación de caños para
mejorar el intercambio de agua dulce y salada.",
    "tipo": "restauracion",
    "municipio": "Pueblo Viejo",
    "departamento": "Magdalena",
    "latitud": 10.991,
    "longitud": -74.282,
    "fechaInicio": "2026-02-01",
    "fechaFin": "2027-01-31",
    "presupuesto": 450000000,
    "estado": "ejecucion",
    "responsableId": 2,
    "responsableNombre": "Roycer Cervantes",
    "creadoEn": "2026-10-23 03:44:05",
    "actualizadoEn": "2026-10-23 03:44:05",
    "impactos": [
      {
        "id": 1,
        "proyectoId": 1,
        "componente": "flora",
        "descripcion": "Recuperación de cobertura de manglar",
        "naturaleza": "positivo",
        "magnitud": 8,
        "probabilidad": 9,
        "importancia": 72,
        "nivel": "critico",
        "medidaManejo": "Seguimiento trimestral de supervivencia de plántulas",
        "creadoEn": "2026-10-23 03:44:05",
        "actualizadoEn": "2026-10-23 03:44:05"
      },
      "... (2 registro(s) más)"
    ],
    "actividades": [
      {
        "id": 1,
        "proyectoId": 1,
        "titulo": "Recolección de propágulos de mangle rojo",
        "descripcion": null,
```

```
        "fechaLimite": "2026-03-15",
        "estado": "completada",
        "responsableId": 2,
        "responsableNombre": "Roycer Cervantes",
        "creadoEn": "2026-10-23 03:44:05",
        "actualizadoEn": "2026-10-23 03:44:05",
        "vencida": false
      },
      "... (2 registro(s) más)"
    ],
    "monitoreos": [
      {
        "id": 2,
        "proyectoId": 1,
        "parametro": "Oxígeno disuelto",
        "valor": 3.1,
        "unidad": "mg/L",
        "limitePermisible": 4,
        "tipoLimite": "minimo",
        "cumpleNorma": false,
        "puntoMuestreo": "Boca de la Barra",
        "fechaMuestreo": "2026-08-20",
        "observaciones": "Valor bajo, posible proliferación de algas",
        "creadoEn": "2026-10-23 03:44:05"
      },
      "... (1 registro(s) más)"
    ]
  }
}
```


Ejemplo de error (400)

```
GET /api/proyectos/abc

{
  "ok": false,
  "mensaje": "El parámetro id debe ser un número entero positivo"
}
```

7.4.4 Actualizar proyecto

Método	PUT
URL	/api/proyectos/:id
Descripción	Modifica los campos enviados. Valida que la fecha de fin no quede antes de la de inicio combinando lo nuevo con lo ya guardado.
Acceso	Token · admin o responsable del proyecto
Parámetros	Body: cualquiera de los campos de creación.
Respuesta exitosa	200

Errores posibles	400 · 403 no es el responsable · 404

Ejemplo de petición

```
PUT /api/proyectos/4
{
  "presupuesto": 135000000,
  "descripcion": "Siembra de 15.000 árboles nativos en zonas de recarga hídrica."
}
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Proyecto actualizado correctamente",
  "datos": {
    "id": 4,
    "codigo": "PIA-2026-0004",
    "nombre": "Reforestación de la cuenca alta del río Gaira",
    "descripcion": "Siembra de 15.000 árboles nativos en zonas de recarga hídrica.",
    "tipo": "restauracion",
    "municipio": "Santa Marta",
    "departamento": "Magdalena",
    "latitud": 11.112,
    "longitud": -74.071,
    "fechaInicio": "2026-11-01",
    "fechaFin": "2027-10-31",
    "presupuesto": 135000000,
    "estado": "planeacion",
    "responsableId": 2,
    "responsableNombre": "Roycer Cervantes",
    "creadoEn": "2026-10-23 03:44:07",
    "actualizadoEn": "2026-10-23 03:44:07"
  }
}
```

Ejemplo de error (403)

```
PUT /api/proyectos/4
{"presupuesto":1}

{
  "ok": false,
  "mensaje": "Solo el responsable del proyecto o un administrador pueden modificarlo"
}
```

7.4.5 Cambiar estado del proyecto

Método	PATCH
URL	/api/proyectos/:id/estado
Descripción	Cambia entre planeación, ejecución, finalizado y suspendido. Un proyecto finalizado solo lo reabre el admin.

Acceso	Token · admin o responsable
Parámetros	Body: estado.
Respuesta exitosa	200
Errores posibles	400 estado inválido o igual al actual · 403 · 404

Ejemplo de petición

```
PATCH /api/proyectos/4/estado
{
  "estado": "ejecucion"
}
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Estado del proyecto cambiado a ejecucion",
  "datos": {
    "id": 4,
    "codigo": "PIA-2026-0004",
    "nombre": "Reforestación de la cuenca alta del río Gaira",
    "descripcion": "Siembra de 15.000 árboles nativos en zonas de recarga hídrica.",
    "tipo": "restauracion",
    "municipio": "Santa Marta",
    "departamento": "Magdalena",
    "latitud": 11.112,
    "longitud": -74.071,
    "fechaInicio": "2026-11-01",
    "fechaFin": "2027-10-31",
    "presupuesto": 135000000,
    "estado": "ejecucion",
    "responsableId": 2,
    "responsableNombre": "Roycer Cervantes",
    "creadoEn": "2026-10-23 03:44:07",
    "actualizadoEn": "2026-10-23 03:44:07"
  }
}
```

7.4.6 Eliminar proyecto

Método	DELETE
URL	/api/proyectos/:id
Descripción	Elimina el proyecto. Sus impactos, actividades y monitoreos se borran solos por el ON DELETE CASCADE.
Acceso	Token · admin o responsable
Parámetros	Ruta: id.
Respuesta exitosa	200

Errores posibles	403 · 404

Ejemplo de petición

```
DELETE /api/proyectos/4
Authorization: Bearer <token>
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Proyecto eliminado junto con sus impactos, actividades y monitoreos"
}
```

Ejemplo de error (404)

```
GET /api/proyectos/4

{
  "ok": false,
  "mensaje": "No existe un proyecto con id 4"
}
```

7.5 Impactos ambientales

7.5.1 Listar impactos de un proyecto

Método	GET
URL	/api/proyectos/:proyectoId/impactos
Descripción	Lista los impactos ordenados de mayor a menor importancia.
Acceso	Token (cualquier rol)
Parámetros	Query: componente, nivel, naturaleza.
Respuesta exitosa	200
Errores posibles	400 filtro inválido · 404 proyecto

Ejemplo de petición

```
GET /api/proyectos/4/impactos?nivel=critico
Authorization: Bearer <token>
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Impactos del proyecto 4",
  "datos": [
    {
      "id": 8,
      "proyectoId": 4,
```

```
      "componente": "suelo",
      "descripcion": "Erosión por remoción de maleza en pendientes fuertes",
      "naturaleza": "negativo",
      "magnitud": 8,
      "probabilidad": 9,
      "importancia": 72,
      "nivel": "critico",
      "medidaManejo": "Siembra en curvas de nivel y trinchos de contención",
      "creadoEn": "2026-10-23 03:44:07",
      "actualizadoEn": "2026-10-23 03:44:07"
    }
  ],
  "total": 1
}
```


7.5.2 Registrar impacto

Método	POST
URL	/api/proyectos/:proyectoId/impactos
Descripción	Registra un impacto. La API calcula importancia = magnitud × probabilidad y lo clasifica: bajo (≤20), moderado (≤40), alto (≤70), crítico (>70).
Acceso	Token · admin o responsable
Parámetros	Body: componente* (agua, aire, suelo, flora, fauna, paisaje, social), descripcion*, naturaleza* (positivo | negativo), magnitud* (1-10), probabilidad* (1-10), medidaManejo.
Respuesta exitosa	201
Errores posibles	400 · 403 · 404

Ejemplo de petición

```
POST /api/proyectos/4/impactos
{
  "componente": "suelo",
  "descripcion": "Erosión por remoción de maleza en pendientes fuertes",
  "naturaleza": "negativo",
  "magnitud": 8,
  "probabilidad": 9,
  "medidaManejo": "Siembra en curvas de nivel y trinchos de contención"
}
```

Respuesta obtenida (201)


```
{
  "ok": true,
  "mensaje": "Impacto ambiental registrado correctamente",
  "datos": {
    "id": 8,
    "proyectoId": 4,
    "componente": "suelo",
```

```
    "descripcion": "Erosión por remoción de maleza en pendientes fuertes",
    "naturaleza": "negativo",
    "magnitud": 8,
    "probabilidad": 9,
    "importancia": 72,
    "nivel": "critico",
    "medidaManejo": "Siembra en curvas de nivel y trinchos de contención",
    "creadoEn": "2026-10-23 03:44:07",
    "actualizadoEn": "2026-10-23 03:44:07"
  }
}
```


Ejemplo de error (400)


```
POST /api/proyectos/4/impactos
{"componente":"ruido","descripcion":"x","naturaleza":"neutro","magnitud":15,"probabilida
d":0}

{
  "ok": false,
  "mensaje": "Hay datos inválidos en la petición",
  "errores": [
    {
      "campo": "componente",
      "mensaje": "El campo componente solo acepta: agua, aire, suelo, flora, fauna,
paisaje, social"
    },
    {
      "campo": "descripcion",
      "mensaje": "El campo descripcion debe tener mínimo 5 caracteres"
    },
    {
      "campo": "naturaleza",
      "mensaje": "El campo naturaleza solo acepta: positivo, negativo"
    },
    {
      "campo": "magnitud",
      "mensaje": "El campo magnitud debe ser menor o igual a 10"
    },
    {
      "campo": "probabilidad",
      "mensaje": "El campo probabilidad debe ser mayor o igual a 1"
    }
  ]
}
```


7.5.3 Consultar impacto

Método	GET
URL	/api/impactos/:id
Descripción	Muestra un impacto.

Acceso	Token (cualquier rol)
Parámetros	Ruta: id.
Respuesta exitosa	200
Errores posibles	404

Ejemplo de petición

```
GET /api/impactos/8
Authorization: Bearer <token>
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Impacto encontrado",
  "datos": {
    "id": 8,
    "proyectoId": 4,
    "componente": "suelo",
    "descripcion": "Erosión por remoción de maleza en pendientes fuertes",
    "naturaleza": "negativo",
    "magnitud": 8,
    "probabilidad": 9,
    "importancia": 72,
    "nivel": "critico",
    "medidaManejo": "Siembra en curvas de nivel y trinchos de contención",
    "creadoEn": "2026-10-23 03:44:07",
    "actualizadoEn": "2026-10-23 03:44:07"
  }
}
```

Ejemplo de error (404)

```
GET /api/impactos/8

{
  "ok": false,
  "mensaje": "No existe un impacto ambiental con id 8"
}
```

7.5.4 Actualizar impacto

Método	PUT
URL	/api/impactos/:id
Descripción	Modifica el impacto. Si cambia la magnitud o la probabilidad, se recalculan importancia y nivel (en el ejemplo bajó de crítico a alto).
Acceso	Token · admin o responsable
Parámetros	Body: cualquiera de los campos de creación.

Respuesta exitosa	200
Errores posibles	400 · 403 · 404

Ejemplo de petición

```
PUT /api/impactos/8
{
  "magnitud": 5
}
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Impacto ambiental actualizado correctamente",
  "datos": {
    "id": 8,
    "proyectoId": 4,
    "componente": "suelo",
    "descripcion": "Erosión por remoción de maleza en pendientes fuertes",
    "naturaleza": "negativo",
    "magnitud": 5,
    "probabilidad": 9,
    "importancia": 45,
    "nivel": "alto",
    "medidaManejo": "Siembra en curvas de nivel y trinchos de contención",
    "creadoEn": "2026-10-23 03:44:07",
    "actualizadoEn": "2026-10-23 03:44:07"
  }
}
```

7.5.5 Eliminar impacto

Método	DELETE
URL	/api/impactos/:id
Descripción	Elimina el impacto.
Acceso	Token · admin o responsable
Parámetros	Ruta: id.
Respuesta exitosa	200
Errores posibles	403 · 404

Ejemplo de petición

```
DELETE /api/impactos/8
Authorization: Bearer <token>
```

Respuesta obtenida (200)

```
{
```

```
  "ok": true,
  "mensaje": "Impacto ambiental eliminado correctamente"
}
```

7.6 Actividades

7.6.1 Listar actividades

Método	GET
URL	/api/proyectos/:proyectoId/actividades
Descripción	Lista las tareas del plan de manejo ordenadas por fecha límite. Cada una trae el campo calculado "vencida".
Acceso	Token (cualquier rol)
Parámetros	Query: estado.
Respuesta exitosa	200
Errores posibles	400 · 404

Ejemplo de petición

```
GET /api/proyectos/4/actividades
Authorization: Bearer <token>
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Actividades del proyecto 4",
  "datos": [
    {
      "id": 8,
      "proyectoId": 4,
      "titulo": "Construcción del vivero temporal",
      "descripcion": "Vivero para 5.000 plántulas de guáimaro y ceiba",
      "fechaLimite": "2026-12-15",
      "estado": "pendiente",
      "responsableId": 2,
      "responsableNombre": "Roycer Cervantes",
      "creadoEn": "2026-10-23 03:44:07",
      "actualizadoEn": "2026-10-23 03:44:07",
      "vencida": false
    }
  ],
  "total": 1
}
```

7.6.2 Crear actividad

Método	POST
URL	/api/proyectos/:proyectoId/actividades
Descripción	Crea una tarea. La fecha límite no puede ser anterior al inicio del proyecto.
Acceso	Token · admin o responsable
Parámetros	Body: titulo*, fechaLimite*, descripcion, estado (pendiente | en_progreso | completada), responsableId.
Respuesta exitosa	201
Errores posibles	400 · 403 · 404

Ejemplo de petición

```
POST /api/proyectos/4/actividades
{
  "titulo": "Construcción del vivero temporal",
  "descripcion": "Vivero para 5.000 plántulas de guáimaro y ceiba",
  "fechaLimite": "2026-12-15",
  "responsableId": 2
}
```

Respuesta obtenida (201)

```
{
  "ok": true,
  "mensaje": "Actividad creada correctamente",
  "datos": {
    "id": 8,
    "proyectoId": 4,
    "titulo": "Construcción del vivero temporal",
    "descripcion": "Vivero para 5.000 plántulas de guáimaro y ceiba",
    "fechaLimite": "2026-12-15",
    "estado": "pendiente",
    "responsableId": 2,
    "responsableNombre": "Roycer Cervantes",
    "creadoEn": "2026-10-23 03:44:07",
    "actualizadoEn": "2026-10-23 03:44:07",
    "vencida": false
  }
}
```

Ejemplo de error (400)

```
POST /api/proyectos/4/actividades
{"titulo":"Tarea con fecha mala","fechaLimite":"2026-01-10"}

{
  "ok": false,
  "mensaje": "Hay datos inválidos en la petición",
  "errores": [
    {
      "campo": "fechaLimite",
```

```
      "mensaje": "La fecha límite no puede ser anterior al inicio del proyecto (2026-11-
01)"
    }
  ]
}
```

7.6.3 Consultar actividad

Método	GET
URL	/api/actividades/:id
Descripción	Muestra una actividad con el nombre de su responsable.
Acceso	Token (cualquier rol)
Parámetros	Ruta: id.
Respuesta exitosa	200
Errores posibles	404

Ejemplo de petición

```
GET /api/actividades/8
Authorization: Bearer <token>
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Actividad encontrada",
  "datos": {
    "id": 8,
    "proyectoId": 4,
    "titulo": "Construcción del vivero temporal",
    "descripcion": "Vivero para 5.000 plántulas de guáimaro y ceiba",
    "fechaLimite": "2026-12-15",
    "estado": "pendiente",
    "responsableId": 2,
    "responsableNombre": "Roycer Cervantes",
    "creadoEn": "2026-10-23 03:44:07",
    "actualizadoEn": "2026-10-23 03:44:07",
    "vencida": false
  }
}
```

7.6.4 Actualizar actividad

Método	PUT
URL	/api/actividades/:id
Descripción	Modifica los campos enviados.

Acceso	Token · admin o responsable
Parámetros	Body: cualquiera de los campos de creación.
Respuesta exitosa	200
Errores posibles	400 · 403 · 404

Ejemplo de petición

```
PUT /api/actividades/8
{
  "fechaLimite": "2026-12-20",
  "estado": "en_progreso"
}
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Actividad actualizada correctamente",
  "datos": {
    "id": 8,
    "proyectoId": 4,
    "titulo": "Construcción del vivero temporal",
    "descripcion": "Vivero para 5.000 plántulas de guáimaro y ceiba",
    "fechaLimite": "2026-12-20",
    "estado": "en_progreso",
    "responsableId": 2,
    "responsableNombre": "Roycer Cervantes",
    "creadoEn": "2026-10-23 03:44:07",
    "actualizadoEn": "2026-10-23 03:44:07",
    "vencida": false
  }
}
```

7.6.5 Cambiar estado de actividad

Método	PATCH
URL	/api/actividades/:id/estado
Descripción	Mueve la actividad entre pendiente, en progreso y completada.
Acceso	Token · admin o responsable
Parámetros	Body: estado.
Respuesta exitosa	200
Errores posibles	400 · 403 · 404

Ejemplo de petición

```
PATCH /api/actividades/8/estado
{
```

```
  "estado": "completada"
}
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Actividad marcada como completada",
  "datos": {
    "id": 8,
    "proyectoId": 4,
    "titulo": "Construcción del vivero temporal",
    "descripcion": "Vivero para 5.000 plántulas de guáimaro y ceiba",
    "fechaLimite": "2026-12-20",
    "estado": "completada",
    "responsableId": 2,
    "responsableNombre": "Roycer Cervantes",
    "creadoEn": "2026-10-23 03:44:07",
    "actualizadoEn": "2026-10-23 03:44:07",
    "vencida": false
  }
}
```

7.6.6 Eliminar actividad

Método	DELETE
URL	/api/actividades/:id
Descripción	Elimina la actividad.
Acceso	Token · admin o responsable
Parámetros	Ruta: id.
Respuesta exitosa	200
Errores posibles	403 · 404

Ejemplo de petición

```
DELETE /api/actividades/8
Authorization: Bearer <token>
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Actividad eliminada correctamente"
}
```

7.7 Monitoreos

7.7.1 Listar monitoreos

Método	GET
URL	/api/proyectos/:proyectoId/monitoreos
Descripción	Lista las mediciones del proyecto, de la más reciente a la más antigua.
Acceso	Token (cualquier rol)
Parámetros	Query: parametro, cumpleNorma (true/false), desde, hasta (AAAA-MM-DD).
Respuesta exitosa	200
Errores posibles	400 fechas mal escritas · 404

Ejemplo de petición

```
GET /api/proyectos/4/monitoreos?cumpleNorma=false
Authorization: Bearer <token>
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Monitoreos del proyecto 4",
  "datos": [
    {
      "id": 6,
      "proyectoId": 4,
      "parametro": "Oxígeno disuelto",
      "valor": 3.2,
      "unidad": "mg/L",
      "limitePermisible": 4,
      "tipoLimite": "minimo",
      "cumpleNorma": false,
      "puntoMuestreo": "Quebrada Mamatoco",
      "fechaMuestreo": "2026-10-20",
      "observaciones": "Muestreo después de lluvias fuertes",
      "creadoEn": "2026-10-23 03:44:07"
    }
  ],
  "total": 1
}
```

7.7.2 Registrar monitoreo

Método	POST
URL	/api/proyectos/:proyectoId/monitoreos
Descripción	Registra una medición. Si se envía el límite permisible, la API decide si cumple: con tipoLimite "maximo" el valor no puede pasarlo (DBO5, coliformes) y con "minimo" no puede quedar por debajo (oxígeno disuelto). Si no cumple, el mensaje lo advierte.

Acceso	Token · admin o responsable
Parámetros	Body: parametro*, valor*, unidad*, fechaMuestreo* (no futura), limitePermisible, tipoLimite, puntoMuestreo, observaciones.
Respuesta exitosa	201
Errores posibles	400 · 403 · 404

Ejemplo de petición

```
POST /api/proyectos/4/monitoreos
{
  "parametro": "Oxígeno disuelto",
  "valor": 3.2,
  "unidad": "mg/L",
  "limitePermisible": 4,
  "tipoLimite": "minimo",
  "puntoMuestreo": "Quebrada Mamatoco",
  "fechaMuestreo": "2026-10-20",
  "observaciones": "Muestreo después de lluvias fuertes"
}
```

Respuesta obtenida (201)

```
{
  "ok": true,
  "mensaje": "Monitoreo registrado. Atención: el valor está FUERA de la norma",
  "datos": {
    "id": 6,
    "proyectoId": 4,
    "parametro": "Oxígeno disuelto",
    "valor": 3.2,
    "unidad": "mg/L",
    "limitePermisible": 4,
    "tipoLimite": "minimo",
    "cumpleNorma": false,
    "puntoMuestreo": "Quebrada Mamatoco",
    "fechaMuestreo": "2026-10-20",
    "observaciones": "Muestreo después de lluvias fuertes",
    "creadoEn": "2026-10-23 03:44:07"
  }
}
```

Ejemplo de error (400)

```
POST /api/proyectos/4/monitoreos
{"parametro":"pH","valor":7,"unidad":"unidades","fechaMuestreo":"2030-01-01"}

{
  "ok": false,
  "mensaje": "Hay datos inválidos en la petición",
  "errores": [
    {
      "campo": "fechaMuestreo",
```

```
      "mensaje": "La fecha de muestreo no puede ser una fecha futura"
    }
  ]
}
```

7.7.3 Consultar monitoreo

Método	GET
URL	/api/monitoreos/:id
Descripción	Muestra una medición.
Acceso	Token (cualquier rol)
Parámetros	Ruta: id.
Respuesta exitosa	200
Errores posibles	404

Ejemplo de petición

```
GET /api/monitoreos/6
Authorization: Bearer <token>
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Monitoreo encontrado",
  "datos": {
    "id": 6,
    "proyectoId": 4,
    "parametro": "Oxígeno disuelto",
    "valor": 3.2,
    "unidad": "mg/L",
    "limitePermisible": 4,
    "tipoLimite": "minimo",
    "cumpleNorma": false,
    "puntoMuestreo": "Quebrada Mamatoco",
    "fechaMuestreo": "2026-10-20",
    "observaciones": "Muestreo después de lluvias fuertes",
    "creadoEn": "2026-10-23 03:44:07"
  }
}
```

7.7.4 Actualizar monitoreo

Método	PUT
URL	/api/monitoreos/:id
Descripción	Modifica la medición y vuelve a evaluar el cumplimiento (en el ejemplo el

	oxígeno subió a 5,6 mg/L y pasó a cumplir).
Acceso	Token · admin o responsable
Parámetros	Body: cualquiera de los campos de creación.
Respuesta exitosa	200
Errores posibles	400 · 403 · 404

Ejemplo de petición

```
PUT /api/monitoreos/6
{
  "valor": 5.6,
  "observaciones": "Segundo muestreo, ya en norma"
}
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Monitoreo actualizado correctamente",
  "datos": {
    "id": 6,
    "proyectoId": 4,
    "parametro": "Oxígeno disuelto",
    "valor": 5.6,
    "unidad": "mg/L",
    "limitePermisible": 4,
    "tipoLimite": "minimo",
    "cumpleNorma": true,
    "puntoMuestreo": "Quebrada Mamatoco",
    "fechaMuestreo": "2026-10-20",
    "observaciones": "Segundo muestreo, ya en norma",
    "creadoEn": "2026-10-23 03:44:07"
  }
}
```

7.7.5 Eliminar monitoreo

Método	DELETE
URL	/api/monitoreos/:id
Descripción	Elimina la medición.
Acceso	Token · admin o responsable
Parámetros	Ruta: id.
Respuesta exitosa	200
Errores posibles	403 · 404

Ejemplo de petición

```
DELETE /api/monitoreos/6
Authorization: Bearer <token>
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Monitoreo eliminado correctamente"
}
```

7.8 Reportes

7.8.1 Resumen general (Dashboard)

Método	GET
URL	/api/reportes/dashboard
Descripción	Totales para las tarjetas del Dashboard y la gráfica ImpactChart: proyectos por estado, impactos por nivel y por componente, actividades vencidas, monitoreos fuera de norma y presupuesto total.
Acceso	Token (cualquier rol)
Parámetros	Ninguno
Respuesta exitosa	200
Errores posibles	401

Ejemplo de petición

```
GET /api/reportes/dashboard
Authorization: Bearer <token>
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Resumen general de la plataforma",
  "datos": {
    "totalProyectos": 4,
    "proyectosPorEstado": {
      "planeacion": 1,
      "ejecucion": 3,
      "finalizado": 0,
      "suspendido": 0
    },
    "impactosPorNivel": {
      "bajo": 0,
      "moderado": 3,
      "alto": 3,
      "critico": 2
    },
    "impactosPorComponente": {
```

```
      "agua": 3,
      "aire": 1,
      "suelo": 2,
      "flora": 1,
      "fauna": 0,
      "paisaje": 0,
      "social": 1
    },
    "actividadesVencidas": 2,
    "monitoreosFueraDeNorma": 3,
    "presupuestoTotal": 2470000000
  }
}
```

7.8.2 Resumen de un proyecto

Método	GET
URL	/api/reportes/proyectos/:id
Descripción	Impactos por nivel, actividades por estado con porcentaje de avance (completadas / total) y monitoreos que cumplen y no cumplen.
Acceso	Token (cualquier rol)
Parámetros	Ruta: id.
Respuesta exitosa	200
Errores posibles	404

Ejemplo de petición

```
GET /api/reportes/proyectos/4
Authorization: Bearer <token>
```

Respuesta obtenida (200)

```
{
  "ok": true,
  "mensaje": "Resumen del proyecto PIA-2026-0004",
  "datos": {
    "proyecto": {
      "id": 4,
      "codigo": "PIA-2026-0004",
      "nombre": "Reforestación de la cuenca alta del río Gaira",
      "estado": "ejecucion"
    },
    "impactosPorNivel": {
      "bajo": 0,
      "moderado": 0,
      "alto": 1,
      "critico": 0
    },
```

```
    "actividades": {
      "pendiente": 0,
      "en_progreso": 0,
      "completada": 1,
      "total": 1,
      "vencidas": 0,
      "porcentajeAvance": 100
    },
    "monitoreos": {
      "total": 1,
      "cumplen": 1,
      "noCumplen": 0
    }
  }
}
```

8. Verificación de los servicios

Para comprobar que todo funcionara antes de entregar, hicimos el script pruebas/probarApi.js. Este script recorre los servicios en orden (registra un usuario, inicia sesión con cada rol, crea un proyecto, le agrega impactos, actividades y monitoreos, consulta los reportes y al final borra todo) y verifica que cada petición devuelva el código HTTP esperado. No solo probamos los casos buenos: también mandamos datos malos a propósito (correos inválidos, magnitudes de 15, fechas futuras, JSON mal escrito, usuarios sin permiso) para confirmar que la API los rechaza con un mensaje claro.

Resultado: 58 de 58 pruebas aprobadas.

Grupo	Pruebas	Casos exitosos (2xx)	Casos de error esperados (4xx)
Salud	1	1	0
Registro	3	1	2
Login	6	4	2
Perfil	2	1	1
Token	1	0	1
Usuarios	8	6	2
Proyectos	13	7	6
Impactos	7	5	2
Actividades	7	6	1
Monitoreos	6	5	1
Reportes	2	2	0
Errores generales	2	0	2

Grupo	Pruebas	Casos exitosos (2xx)	Casos de error esperados (4xx)

Las pruebas formales con Postman, con su video y capturas, corresponden a la siguiente evidencia (AA5-EV04). Para ese trabajo ya quedan listos los endpoints, los datos de prueba y los ejemplos de esta documentación.

9. Instalación y ejecución

Requisitos: Node.js 18 o superior y MySQL 8 (sirve el de XAMPP).

```
# 1. Clonar el repositorio
git clone https://github.com/royzeer/plataforma-impacto-ambiental-api.git
cd plataforma-impacto-ambiental-api

# 2. Instalar dependencias
npm install

# 3. Configurar la conexión (copiar y editar con los datos de MySQL)
copy .env.example .env

# 4. Crear la base de datos
mysql -u root -p < database/schema.sql

# 5. Cargar datos de prueba
npm run seed

# 6. Levantar la API
npm run dev

# 7. (Opcional) Probar todos los servicios, en otra terminal
npm run probar
```

Con la API arriba, la documentación interactiva queda en http://localhost:3000/api/docs. Los usuarios de prueba son admin@plataforma.com, roycer.cervantes@plataforma.com, sebastian.rodriguez@plataforma.com y consultor@plataforma.com, todos con la contraseña Ambiental2026.

10. Control de versiones

El proyecto se trabajó con Git desde el inicio y se publicó en GitHub. Cada parte de la API quedó en un commit separado (estructura, base de datos, autenticación, cada módulo, pruebas y documentación), lo que permite ver cómo fue creciendo el proyecto y volver a una versión anterior si algo se daña. El archivo .gitignore deja por fuera node_modules y el .env con las contraseñas.

Repositorio: **https://github.com/royzeer/plataforma-impacto-ambiental-api**

11. Conclusiones

- Pasar del servicio de login de la EV01 a una API completa nos obligó a pensar primero en el diseño: qué recursos existen, cómo se relacionan y quién puede hacer qué. Cuando tuvimos clara la tabla de permisos, el código salió mucho más ordenado.
- Separar rutas, controladores y modelos fue clave. Al principio parecía más trabajo crear tantos archivos, pero a la hora de corregir errores sabíamos exactamente dónde buscar.
- Dejar que la API calcule la importancia de los impactos y el cumplimiento de la norma evita errores humanos y conecta lo que aprendemos en Ingeniería Ambiental con el desarrollo de software. Un caso que nos gustó fue el del oxígeno disuelto, que tiene límite mínimo y no máximo como casi todos los parámetros.
- Probar también los casos de error fue tan importante como probar los casos buenos: varias validaciones (como la fecha límite anterior al inicio del proyecto) salieron justamente de pensar "¿y si el usuario manda esto?".
- Con la API documentada en Swagger y en este informe, el front en React ya puede consumir datos reales, y la siguiente evidencia (pruebas con Postman) parte de una base estable.

12. Referencias

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Tesis doctoral, Universidad de California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Jones, M., Bradley, J. y Sakimura, N. (2015). JSON Web Token (JWT) (RFC 7519). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7519

Mozilla Developer Network. (s. f.). Códigos de estado de respuesta HTTP. https://developer.mozilla.org/es/docs/Web/HTTP/Status

OpenAPI Initiative. (2021). OpenAPI Specification v3.0.3. https://spec.openapis.org/oas/v3.0.3

OpenJS Foundation. (s. f.). Express: Framework web rápido, minimalista y flexible para Node.js. https://expressjs.com/es/

Oracle. (s. f.). MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

Servicio Nacional de Aprendizaje SENA. (2022). Componente formativo: Construcción de API. Programa Análisis y Desarrollo de Software.

Servicio Nacional de Aprendizaje SENA. (2024). Guía de aprendizaje 7: Construcción de software integrador de tecnologías orientadas a servicios (GFPI-F-135 V02).